

Machine Learning A (2026)
Home Assignment 5

Casper Moes (hwd362)

Contents

1	On the Role of Dependence	2
2	On Confidence Intervals	2
2.1	Part 1	2
2.2	Part 2	3
3	Loss Range Correction in Generalization Bounds	4
3.1	(i)	4
3.2	(ii)	4
4	Convolutional Neural Networks	5
4.1	Sobel filter	5
4.2	Convolutional neural networks	6
4.3	Augmentation	7

1 On the Role of Dependence

Let $X_i \in \{0, 1\}$, $i \in \{1, 2, \dots, n\}$ be *completely* dependent stochastic variables, by which i mean

$$\mathbb{P}(X_1 = X_2 = \dots = X_n) = 1,$$

which implies that

$$\frac{1}{n} \sum_{i=1}^n X_i = \frac{1}{n} \sum_{i=1}^n X_1 = X_1 \in \{0, 1\}.$$

Let the *bias* or *success probability* of the all the bernoulli random variables be $p = \frac{1}{2}$ (i.e. $\mathbb{P}(X_i = 1) = \mathbb{P}(X_i = 0) = \frac{1}{2}$ for all i). Then calculating the expected value of X_1 (or any X_i) yields

$$\mu \equiv \mathbb{E}[X_1] = \mathbb{P}(X_1 = 1) = \frac{1}{2},$$

and thus we know that $|\mu - \frac{1}{n} \sum_{i=1}^n X_i| \in \{\frac{1}{2}, \frac{1}{2}\}$ and can complete the argument:

$$\left| \mu - \frac{1}{n} \sum_{i=1}^n X_i \right| = \left| \frac{1}{2} - X_1 \right| = \frac{1}{2} \geq \frac{1}{2} \implies \mathbb{P} \left(\left| \mu - \frac{1}{n} \sum_{i=1}^n X_i \right| \geq \frac{1}{2} \right) = 1$$

2 On Confidence Intervals

2.1 Part 1

Firstly we remember the definition of a confidence interval for θ as the function of the datapoints $CI(X_1, X_2, \dots, X_n)$ fulfilling

$$\mathbb{P}(\theta \in CI(X_1, X_2, \dots, X_n)) \geq 1 - \alpha, \tag{1}$$

where α is the significance level. With $\alpha = 0.02$ we get the 98%– CI that the assignment revolves around.

1. **False:** The confidence does not generally try to encompass the observed variables by some margin (this is also the explanation for question 2). The confidence interval here concerns coverage of the population mean across repeated samples; it does not specify the fraction of observations inside the interval.

2. **False:** See 1.

3. **False:** By the definition of a confidence interval (i.e. (1)) CI is a stochastic variable that depending on the realisation of $X_1, X_2, \dots, X_n$ and thus is generally not the same between two different draws of data.

4. **True** (in the case of CI based on Hoeffding bound): By the double sided Hoeffding bound we have

$$\mathbb{P} \left(\left| \sum_{i=1}^n X_i - \mathbb{E} \left[\sum_{i=1}^n X_i \right] \right| \leq \sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2} \ln \left(\frac{2}{\delta} \right)} \right) \geq 1 - \delta.$$

This is shown by choosing rewriting the parameters and taking the complement of the original event. Rewriting the event (by scaling with $1/n$) we obtain the following

$$\begin{aligned} \left| \frac{1}{n} \sum_{i=1}^n X_i - \frac{1}{n} \mathbb{E} \left[\sum_{i=1}^n X_i \right] \right| &= |\hat{\mu}_n - \mu| \leq \sqrt{-\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{\delta}{2} \right)} \iff \\ &-\sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{2}{\delta} \right)} \leq \hat{\mu}_n - \mu \leq \sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{2}{\delta} \right)} \iff \\ \hat{\mu}_n + \sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{2}{\delta} \right)} &\geq \mu \geq \hat{\mu}_n - \sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{2}{\delta} \right)} \end{aligned} \quad (2)$$

That is with at least $1 - \delta$ probability the *true* / *population* expected value μ will fulfill (2), which formally mean

$$\mathbb{P} \left(\mu \in \left[\hat{\mu}_n - \sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{2}{\delta} \right)}, \quad \hat{\mu}_n + \sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{2}{\delta} \right)} \right] \right) \geq 1 - \delta, \quad (3)$$

which is the definition of the confidence interval (see (1)). From (3) we see that decreasing $\delta \in [0, 1]$ (for example from 0.02 to 0.01) implies a larger confidence interval, since

$$\frac{d}{d\delta} \left(\ln \left(\frac{1}{\delta} \right) \right) = \frac{d}{d\delta} (-\ln \delta) = -\frac{1}{\delta} < 0,$$

and thus decreasing δ has a positive effect on $\ln(2/\delta)$ and thus $\sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{2}{\delta} \right)}$.

However in general we could define some weird confidence interval procedure that fulfills (1) by setting $I = (-\infty, \infty)$ for $\delta > 0.01$

$$\text{and } I = \left[\hat{\mu}_n - \sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{2}{\delta} \right)}, \quad \hat{\mu}_n + \sqrt{\frac{\sum_{i=1}^n (b_i - a_i)^2}{2n^2} \ln \left(\frac{2}{\delta} \right)} \right] \text{ for } \delta \in [0, 0.01].$$

2.2 Part 2

The requested interval is $[0.23, 0.44]$ since $[0.237, 0.439] \subseteq [0.23, 0.44]$ and therefore we have

$$\{\mu \in [0.237, 0.439]\} \subseteq \{\mu \in [0.23, 0.44]\} \implies \mathbb{P}(\mu \in [0.23, 0.44]) \geq \mathbb{P}(\mu \in [0.237, 0.439]),$$

so the original confidence guarantee is preserved.

In contrast, ordinary rounding would give $[0.24, 0.44]$. However, $[0.237, 0.439] \not\subseteq [0.24, 0.44]$, since the values in $[0.237, 0.24)$ are excluded. Hence, $\{\mu \in [0.237, 0.439]\} \not\subseteq \{\mu \in [0.24, 0.44]\}$, and we therefore cannot guarantee that

$$\mathbb{P}(\mu \in [0.24, 0.44]) \geq \mathbb{P}(\mu \in [0.237, 0.439]).$$

Thus, $[0.23, 0.44]$ is the tightest interval with endpoints reported to two decimal places for which the original confidence guarantee is certainly preserved.

3 Loss Range Correction in Generalization Bounds

3.1 (i)

No. The reported bound is based on the assumption that the loss is bounded in $[0, 1]$. Hoeffding's inequality depends on the width of the range, $b - a$. The assumed width was therefore 1, whereas the correct width is

$$1.44 - 0.18 = 1.26.$$

Since the true range is wider, the Hoeffding bound used to obtain $L(h) \leq 0.38$ does not provide the claimed 95% guarantee.

3.2 (ii)

Yes. For losses bounded in $[a, b]$, the one-sided Hoeffding bound gives, with probability at least $1 - \delta$,

$$L(h) \leq \widehat{L}_S(h) + (b - a) \sqrt{\frac{\ln(1/\delta)}{2n}}.$$

The original bound used $b - a = 1$, $n = 1000$ and $\delta = 0.05$, so

$$0.38 = \widehat{L}_S(h) + \sqrt{\frac{\ln(20)}{2000}}.$$

Hence we can correct the bound without observing S again. Using the correct range width 1.26 gives

$$L(h) \leq \widehat{L}_S(h) + 1.26 \sqrt{\frac{\ln(20)}{2000}} = 0.38 + 0.26 \sqrt{\frac{\ln(20)}{2000}} \approx 0.390.$$

Thus the corrected generalization bound is $L(h) \leq 0.390$ with probability at least 0.95.

4 Convolutional Neural Networks

4.1 Sobel filter

Firstly we note that the Sobel filter / kernel is 3×3 . Therefore, to preserve the dimensions of the input space through to the feature space, we pad the entire picture so that the padded input is an array with dimension $(480 + 2 \times 640 + 2)$ where the original input has dimension 480×640 . Here 0-padding is used.

The Sobel filter contains two steps. Firstly the following two convolutions are applied with stride 1

$$\mathcal{G}_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} * \mathbf{I} \quad \mathcal{G}_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{I},$$

where $\mathbf{I}$ is the input image. I implement this in python using PyTorch as follows

```
# "vertical" kernel
v = torch.tensor([[[[1., 0., -1.],
                    [2., 0., -2.],
                    [1., 0., -1.]]]])

# "horizontal" kernel
h = torch.tensor([[[[ 1., 2., 1.],
                    [ 0., 0., 0.],
                    [-1., -2., -1.]]]])

# compute the first convolution step of the Sobel filter
kernels = torch.cat([v, h], dim=0)
kernels = kernels.flip(dims=(-2, -1))

conv = nn.Conv2d(1, 2, kernel_size=3, padding=1, bias=False)
conv.weight = nn.Parameter(kernels, requires_grad=False)

maps = conv(x)
G_x = maps[:, 0:1]
G_y = maps[:, 1:2]
```

The stored intermediary feature maps are shown in figure 1 where their purpose it clearly visible. $\mathcal{G}_x$ highlights the horizontal edges in $\mathbf{I}$ and $\mathcal{G}_y$ the vertical. We combine the two mappings into the Sobel filter

$$\mathcal{G} = \begin{bmatrix} \sqrt{[\mathcal{G}_x]_{11}^2 + [\mathcal{G}_y]_{11}^2} & \sqrt{[\mathcal{G}_x]_{12}^2 + [\mathcal{G}_y]_{12}^2} & \cdots & \sqrt{[\mathcal{G}_x]_{1640}^2 + [\mathcal{G}_y]_{1640}^2} \\ \sqrt{[\mathcal{G}_x]_{21}^2 + [\mathcal{G}_y]_{21}^2} & \sqrt{[\mathcal{G}_x]_{22}^2 + [\mathcal{G}_y]_{22}^2} & \cdots & \sqrt{[\mathcal{G}_x]_{2640}^2 + [\mathcal{G}_y]_{2640}^2} \\ \vdots & \vdots & \ddots & \vdots \\ \sqrt{[\mathcal{G}_x]_{4801}^2 + [\mathcal{G}_y]_{4801}^2} & \sqrt{[\mathcal{G}_x]_{4802}^2 + [\mathcal{G}_y]_{4802}^2} & \cdots & \sqrt{[\mathcal{G}_x]_{480640}^2 + [\mathcal{G}_y]_{480640}^2} \end{bmatrix},$$

which i implement i Python like this

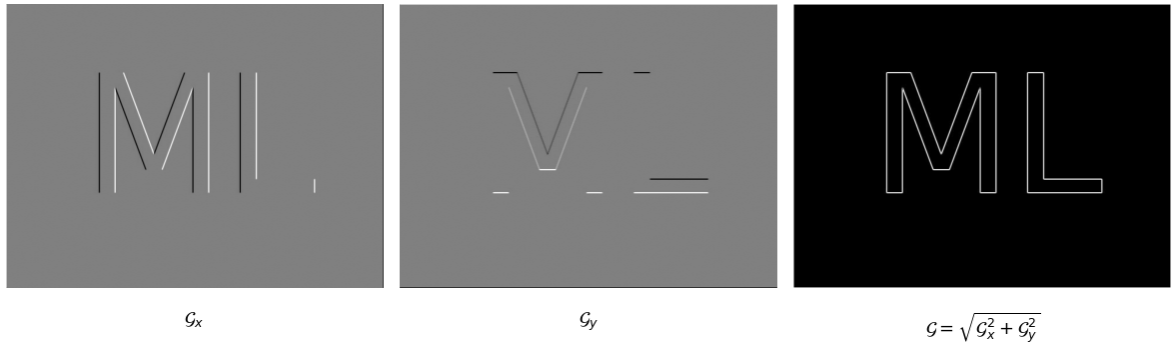


Figure 1: Sobel filter with intermediary steps

Note: The figure plots the Sobel filter and the intermediary steps as explained in section 4.1.

```
# calculate the Sobel filter
G = torch.sqrt(G_x**2 + G_y**2)
```

The resulting feature map is the right most plot in figure 1 and is very intuitive: It is simply the combined horizontal and vertical filters with positive output-values due to squaring.

PyTorch's `nn.Conv2d` computes cross-correlation, whereas convolution rotates the kernel by 180° . I use the Sobel kernels without rotating them, which reverses the signs of G_x and G_y compared with convolution. However, this does not affect the final result $G = \sqrt{G_x^2 + G_y^2}$, since both feature maps are squared.

4.2 Convolutional neural networks

I define the model class as using PyTorch as the assignemnt reccomends. I define the two convolutional layers: the first takes 3 inputlayers (i.e. the RGB values of the pictures) and outputs 64 feature maps. The kernel is 5×5 so the feature map is now a square with sidelengths both equal to $28 - 5 + 1 = 24$, and we apply the ELU activation function. This is pooled using a 2×2 maxpool layer and the dimension is now 12×12 . The second Convolutional layer takes 64 inputs (i.e. the maxpooled feature maps of the first block) and outputs 64 feature maps. Dimension of each feature map is thus $12 - 5 + 1 = 8$. The same pooling is then applied. Finally the $4 \times 4 \times 64 = 1024$ values are flattened into a layer fully connected to 43 output neurons. The `foward` method of the class defines the sequence of operations:

```
class Net(nn.Module):
```

```

def __init__(self):
    super().__init__()

    self.conv1 = nn.Conv2d(3, 64, kernel_size=5)
    self.pool1 = nn.MaxPool2d(2)

    self.conv2 = nn.Conv2d(64, 64, kernel_size=5)
    self.pool2 = nn.MaxPool2d(2)

    self.fc2 = nn.Linear(1024, 43)

def forward(self, x):
    x = self.pool1(F.relu(self.conv1(x)))
    x = self.pool2(F.relu(self.conv2(x)))

    x = x.flatten(1)

    x = self.fc2(x)

    return x

```

The parameters of the network are estimated by minimizing the cross-entropy loss on the training data. For each image, the network produces 43 output scores, or logits, $(s_1, \dots, s_{43})$.

I train the network for 200 epochs using mini-batches of 128 observations. The training observations are shuffled at the beginning of each epoch. For each mini-batch, I first perform a forward pass through the network and compute the cross-entropy loss. PyTorch then uses backpropagation to calculate the gradient of the loss with respect to every trainable parameter in the network, including the convolutional kernels, convolutional biases, and parameters of the fully connected layer. These parameters are subsequently updated using the Adam optimizer with a learning rate of (0.001). This procedure is repeated for every mini-batch and for all 200 epochs.

After training, I evaluate the network on the separate test set containing 12,630 images. For each test image, the predicted class is the class corresponding to the largest of the 43 output scores. The network correctly classifies 11,905 of the 12,630 test images, giving a test accuracy of 94.26%. Since the test observations are not used to train the network parameters, this accuracy is an unbiased estimate of how well the fitted model generalizes.

4.3 Augmentation

The training data are first resized to 32×32 . Four augmentations are then applied: a random rotation between -5° and 5° , a random 28×28 crop, random brightness and contrast changes, and a horizontal flip with probability 0.5. The horizontal flip is conditioned on the label and is only applied to labels 11, 12, 13, 17, 18, 26, 30, and 35. These are all signs that are horizontally symmetrical (i.e. their meaning is preserved under horizontal flipping). For the test data, only a deterministic center crop is used. Finally, all images are converted

to tensors.

As an additional augmentation I use a small random perspective transformation:

```
image = transforms.RandomPerspective(  
    distortion_scale=0.15, p=0.3  
) (image)
```

This is reasonable for traffic-sign recognition since signs are often photographed from a slight angle rather than perfectly front-on. A small perspective distortion can therefore approximate realistic variation in the viewing angle while preserving the class of the sign.